

Translated

From: e-maxx.ru

Ternary Search

We are given a function $f(x)$ which is unimodal on an interval $[l, r]$. By unimodal function, we mean one of two behaviors of the function:

1. The function strictly increases first, reaches a maximum (at a single point or over an interval), and then strictly decreases.
2. The function strictly decreases first, reaches a minimum, and then strictly increases.

In this article, we will assume the first scenario. The second scenario is completely symmetrical to the first.

The task is to find the maximum of function $f(x)$ on the interval $[l, r]$.

Algorithm

Consider any 2 points m_1 , and m_2 in this interval: $l < m_1 < m_2 < r$. We evaluate the function at m_1 and m_2 , i.e. find the values of $f(m_1)$ and $f(m_2)$. Now, we get one of three options:

- $f(m_1) < f(m_2)$

The desired maximum can not be located on the left side of m_1 , i.e. on the interval $[l, m_1]$, since either both points m_1 and m_2 or just m_1 belong to the area where the function increases. In either case, this means that we have to search for the maximum in the segment $[m_1, r]$.

- $f(m_1) > f(m_2)$

This situation is symmetrical to the previous one: the maximum can not be located on the right side of m_2 , i.e. on the interval $[m_2, r]$, and the search space is reduced to the segment $[l, m_2]$.

- $f(m_1) = f(m_2)$

We can see that either both of these points belong to the area where the value of the function is maximized, or m_1 is in the area of increasing values and m_2 is in the area of descending values (here we used the strictness of function increasing/decreasing). Thus, the search space is reduced to $[m_1, m_2]$. To simplify the code, this case can be combined with any of the previous cases.

Thus, based on the comparison of the values in the two inner points, we can replace the current interval $[l, r]$ with a new, shorter interval $[l', r']$. Repeatedly applying the described procedure to the interval, we can get an arbitrarily short interval. Eventually, its length will be less than a certain pre-defined constant (accuracy), and the process can be stopped. This is a numerical method, so we can assume that after that the function reaches its maximum at all points of the last interval $[l, r]$. Without loss of generality, we can take $f(l)$ as the return value.

We didn't impose any restrictions on the choice of points m_1 and m_2 . This choice will define the convergence rate and the accuracy of the implementation. The most common way is to choose the points so that they divide the interval $[l, r]$ into three equal parts. Thus, we have

$$m_1 = l + \frac{(r - l)}{3}$$

$$m_2 = r - \frac{(r - l)}{3}$$

If m_1 and m_2 are chosen to be closer to each other, the convergence rate will increase slightly.

Run time analysis

$$T(n) = T(2n/3) + O(1) = \Theta(\log n)$$

It can be visualized as follows: every time after evaluating the function at points m_1 and m_2 , we are essentially ignoring about one third of the interval, either the left or right one. Thus the size of the search space is $2n/3$ of the original one.

Applying [Master's Theorem](#), we get the desired complexity estimate.

The case of the integer arguments

If $f(x)$ takes integer parameter, the interval $[l, r]$ becomes discrete. Since we did not impose any restrictions on the choice of points m_1 and m_2 , the correctness of the algorithm is not affected. m_1 and m_2 can still be chosen to divide $[l, r]$ into 3 approximately equal parts.

The difference occurs in the stopping criterion of the algorithm. Ternary search will have to stop when $(r - l) < 3$, because in that case we can no longer select m_1 and m_2 to be different from each other as well as from l and r , and this can cause an infinite loop. Once $(r - l) < 3$, the remaining pool of candidate points $(l, l + 1, \dots, r)$ needs to be checked to find the point which produces the maximum value $f(x)$.

Golden section search

In some cases computing $f(x)$ may be quite slow, but reducing the number of iterations is infeasible due to precision issues. Fortunately, it is possible to compute $f(x)$ only once at each iteration (except the first one).

To see how to do this, let's revisit the selection method for m_1 and m_2 . Suppose that we select m_1 and m_2 on $[l, r]$ in such a way that $\frac{r-l}{r-m_1} = \frac{r-l}{m_2-l} = \varphi$ where φ is some constant. In order to reduce the amount of computations, we want to select such φ that on the next iteration one of the new evaluation points m'_1, m'_2 will coincide with either m_1 or m_2 , so that we can reuse the already computed function value.

Now suppose that after the current iteration we set $l = m_1$. Then the point m'_1 will satisfy $\frac{r-m_1}{r-m'_1} = \varphi$. We want this point to coincide with m_2 , meaning that $\frac{r-m_1}{r-m_2} = \varphi$.

Multiplying both sides of $\frac{r-m_1}{r-m_2} = \varphi$ by $\frac{r-m_2}{r-l}$ we obtain $\frac{r-m_1}{r-l} = \varphi \frac{r-m_2}{r-l}$. Note that $\frac{r-m_1}{r-l} = \frac{1}{\varphi}$ and $\frac{r-m_2}{r-l} = \frac{r-l+m_2-m_1}{r-l} = 1 - \frac{1}{\varphi}$. Substituting that and multiplying by φ , we obtain the following equation:

$$\varphi^2 - \varphi - 1 = 0$$

This is a well-known golden section equation. Solving it yields $\frac{1 \pm \sqrt{5}}{2}$. Since φ must be positive, we obtain $\varphi = \frac{1 + \sqrt{5}}{2}$. By applying the same logic to the case when we set $r = m_2$ and want m'_2 to coincide with m_1 , we obtain the same value of φ as well. So, if we choose $m_1 = l + \frac{r-l}{1+\varphi}$ and $m_2 = r - \frac{r-l}{1+\varphi}$, on each iteration we can re-use one of the values $f(x)$ computed on the previous iteration.

Implementation

```
double ternary_search(double l, double r) {
    double eps = 1e-9;           //set the error limit here
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);       //evaluates the function at m1
        double f2 = f(m2);       //evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l);                 //return the maximum of f(x) in [l, r]
}
```

Here `eps` is in fact the absolute error (not taking into account errors due to the inaccurate calculation of the function).

Instead of the criterion `r - l > eps`, we can select a constant number of iterations as a stopping criterion. The number of iterations should be chosen to ensure the required accuracy. Typically, in most programming challenges the error limit is 10^{-6} and thus 200 - 300 iterations are sufficient. Also, the number of iterations doesn't depend on the values of l and r , so the number of iterations corresponds to the required relative error.

Contributors:

likecs (39.32%) Yury Semenov (17.09%) Mariia Mykhailova (16.24%) Abhijeet Krishnan (11.11%)
 Oleksandr Kulkov (5.98%) Morass (5.13%) Joaquin Garmendia Cabrera (1.71%) Ahmed Marouf (0.85%)
 Lovish Madhu (0.85%) Puneet Kumar (0.85%) jxu (0.85%)